

A Fast Algorithm for Generating All Triangulations of Biconnected Plane Graphs

Tawkir Aziz Rahman

CSE, BUET

Overview

- 1 Introduction
- 2 Our Approach
- 3 Algorithm
- 4 Proofs
- 5 Time Complexity Analysis

The Problem:

- Generate **all possible triangulations** of biconnected plane graphs
- Previous methods fail with complex structures due to multi-edge creation
- Important applications in graphics, engineering, and chip design

Our Contribution:

- Novel algorithm with **constant time per triangulation**
- Solves the multi-edge problem using **safe root** concept
- Practical for real-world applications

Our Approach: Key Innovations

Novel Tree Structure:

- Instead of a single global genealogical tree, we use **local genealogical trees** for each face/cycle
- Each face maintains its own tree of triangulations with parent-child relationships
- Enables independent processing of faces while avoiding multi-edge issues

Safe Root Concept:

- Introduces **safe root vertex selection** to prevent multi-edge creation
- Uses incremental chord checking to find optimal root vertex position
- Ensures all initial chords from root vertex are valid and non-conflicting

Our Approach: Recursive Tree Construction

Local Tree Generation:

- For each face F_i , build a genealogical tree $\mathcal{T}(F_i)$
- Initialize with safe root triangulation where all chords emanate from safe root vertex
- Generate children by flipping generating chords while maintaining parent-child relationships

GlobalChordSet Management:

- Maintain global set of all chords used across all faces
- Before adding any chord, check if it already exists in GlobalChordSet
- Prevents multi-edge creation by rejecting conflicting chords
- Dynamic addition/removal during recursive exploration

Our Approach: Combinatorial Integration

Recursive Face Processing:

- Process faces sequentially from F_1 to F_k
- For each partial triangulation T , recursively triangulate remaining faces
- Combine local triangulations T_i to form complete graph triangulation $T + T_i$

Efficient Output Generation:

- Output complete triangulation only when all faces are processed ($i = \text{face.length}$)
- For intermediate states, recursively continue to next face
- Ensures only valid complete triangulations are output

Constant-Time Operations:

- Safe root finding in $O(1)$ time per face chord operations and GlobalChordSet updates in constant time
- Overall $O(1)$ time per triangulation generation

Our Approach: Algorithm Overview

Start with Biconnected Graph

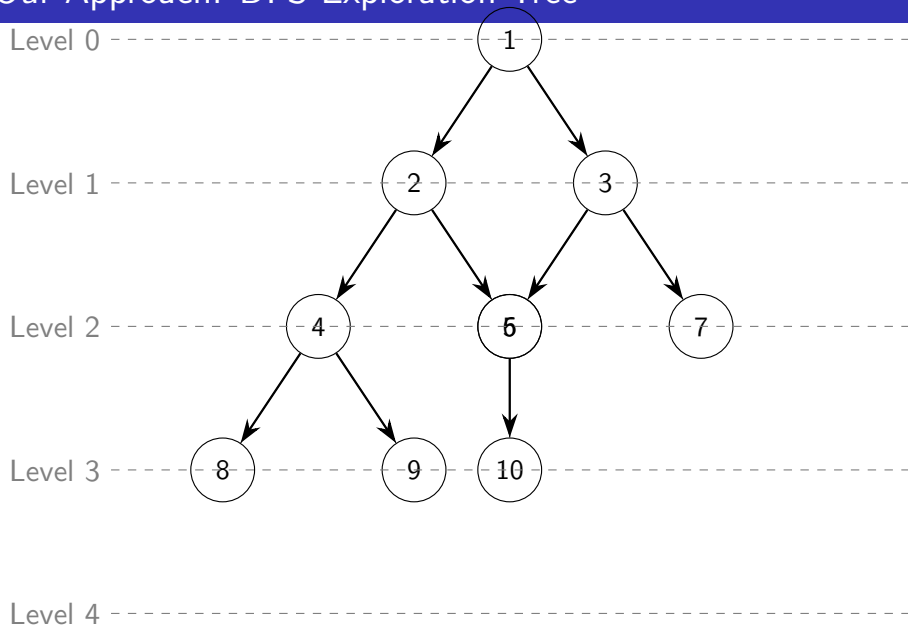


Decompose into Faces $F_1, F_2, \dots, F_k$

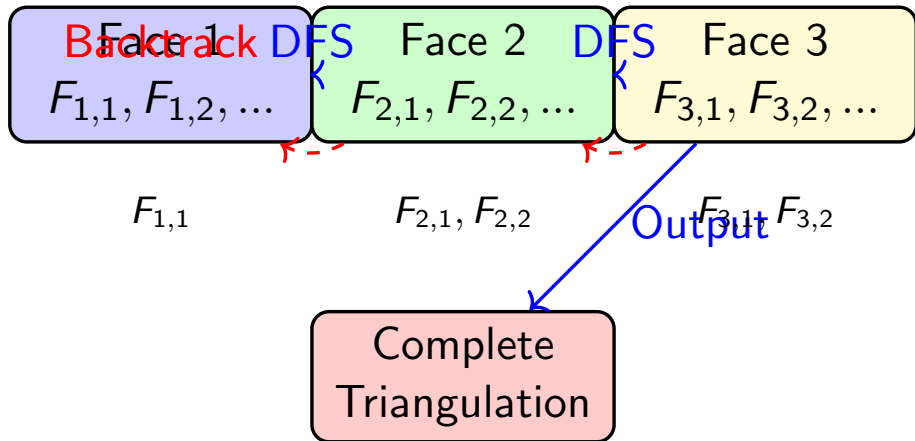


Find Safe Root for Each Face

Our Approach: DFS Exploration Tree

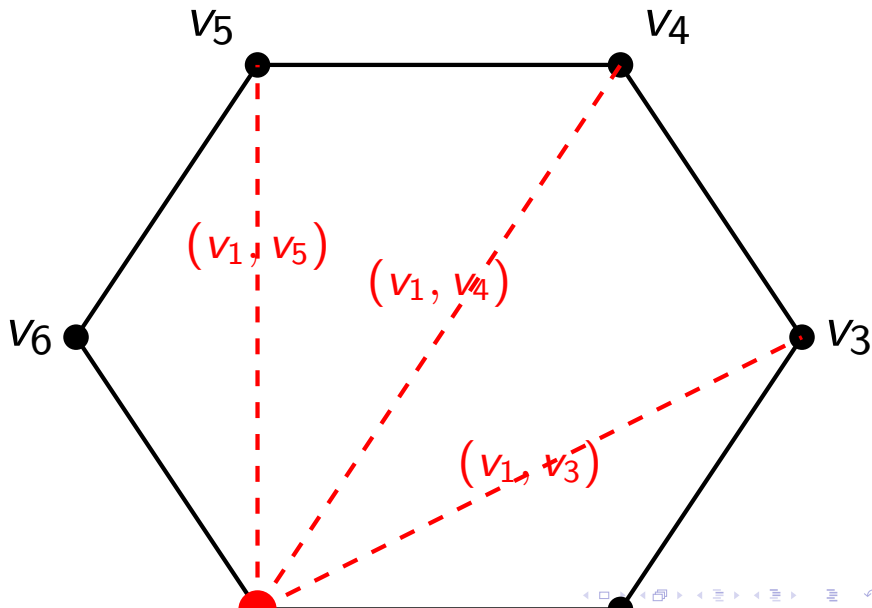


Our Approach: Face Processing Workflow

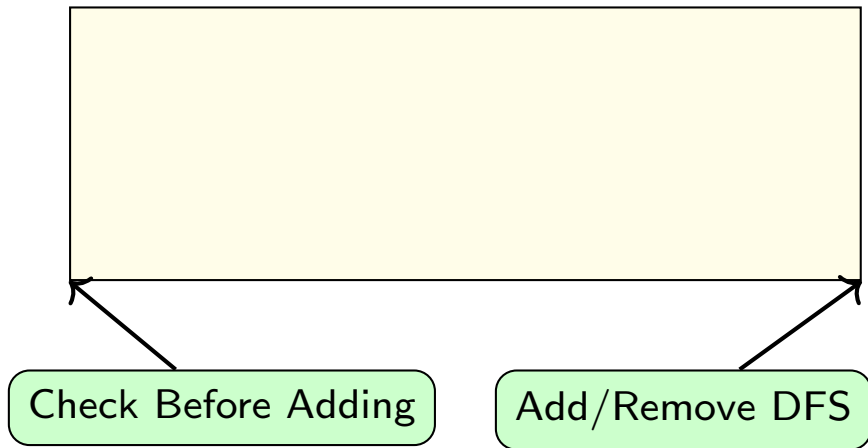


Process: For each triangulation of Face i , recursively find all compatible triangulations for Face $i + 1$

Our Approach: Safe Root Concept Visualization

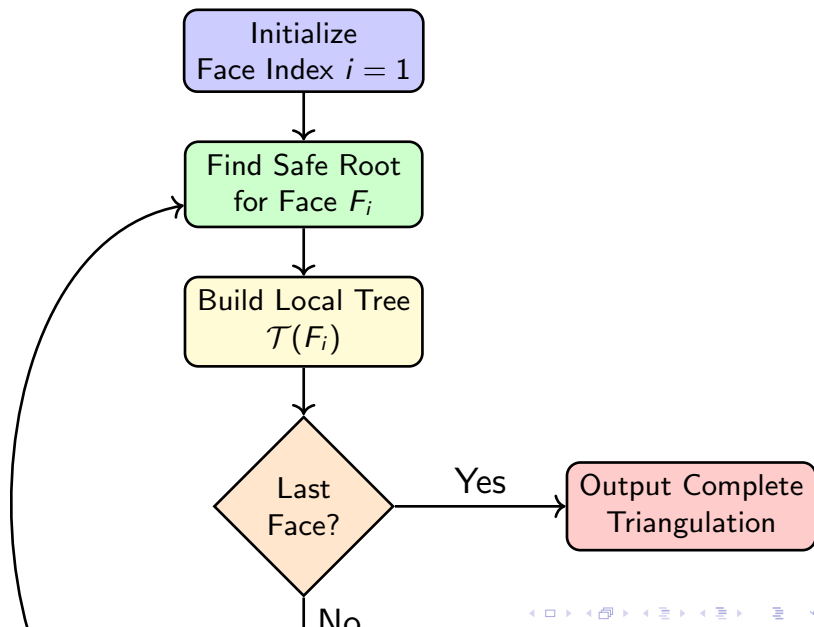


GlobalChordSet



Purpose: Prevents multi-edge creation by tracking all chords used across faces

Our Approach: Complete Algorithm Flow



Find All Triangulations Cycle

```
1: procedure FIND-ALL-CHILD-TRIANGULATIONS-CYCLE( $T_i, T, i$ )
2:   OUTPUT-TRIANGULATION( $T, T_{ir}, i$ )
3:   if  $T_i$  has no generating chords then
4:     return
5:   end if
6:   Let  $(v_b, v_{b'})$  be the leftmost blocking chord of  $T_i$ 
7:   for all  $j \geq b$  do
8:     if  $(v_1, v_j)$  is a chord of  $T_i \vee (v_1, v_j) \notin \text{GlobalChordSet}$  then
9:       add chord  $(v_1, v_j)$  to GlobalChordSet
10:      FIND-ALL-CHILD-TRIANGULATIONS-CYCLE( $T(v_1, v_j)$ )
11:      remove chord  $(v_1, v_j)$  from GlobalChordSet
12:    end if
13:  end for
14: end procedure
```

Find All Triangulations

```
1: procedure FIND-ALL-TRIANGULATIONS-CYCLE( $F, T, i$ )
2:    $n \leftarrow$  number of vertices for  $F_i$ 
3:    $T_{ir} \leftarrow$  FIND-SAFE-ROOT( $F, i$ )                                ▷ Time  $O(1)$ 
4:   Add All Chords of the root to GlobalChordSet                      ▷ Time  $O(n)$ 
5:   OUTPUT-TRIANGULATION( $T, T_{ir}, i$ )
6:    $T_i = T_{ir}$ 
7:   for  $j = n - 1$  to 3 do
8:     add chord  $(v_1, v_j)$  to GlobalChordSet
9:     FIND-ALL-CHILD-TRIANGULATIONS-CYCLE( $T_i(v_1, v_j), T, i$ )
10:    Remove chord  $(v_1, v_j)$  from GlobalChordSet
11:  end for
12:  Remove All Chords of the root From GlobalChordSet                ▷ Time  $O(n)$ 
13: end procedure
```

```
1: procedure OUTPUT-TRIANGULATION( $T, T_i, i$ )
2:   if  $i == \text{face.length}$  then
3:     output( $T + T_i$ )
4:   else
5:     FIND-ALL-TRIANGULATION-CYCLE( $F, T + T_i, i + 1$ )
6:   end if
7: end procedure
```

Find Safe Root

```
1: procedure FIND-SAFE-ROOT( $F, index$ )
2:   FIND-SAFE-ROOT-VERTEX( $F, index$ )
3:   add chords from the safe root vertex
4:   return
5: end procedure
```

```
1: procedure FIND-SAFE-ROOT-VERTEX( $F, index$ )
2:   startindex  $\leftarrow n - 1$ 
3:   endIndex  $\leftarrow 0$ 
4:   while startindex  $>$  endIndex + 1 do
5:     if chord exists between  $V_{startindex}$  and  $V_{endIndex}$  then
6:       startindex  $\leftarrow startindex - 1$ 
7:     else
8:       endIndex  $\leftarrow endIndex + 1$ 
9:     end if
10:  end while
11: end procedure
```


Start Triangulation Process

- 1: **procedure** START-TRIANGULATION(G, k)
- 2: Label the faces $F_1, F_2, \dots, F_k$ arbitrarily
- 3: FIND-ALL-TRIANGULATIONS($F, 0$)
- 4: **end procedure**

Proof Strategy Overview

Three Main Theorems to Prove:

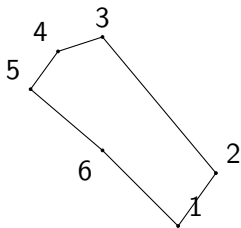
- 1 **Completeness:** Algorithm generates all valid triangulations
- 2 **Safe Root Existence:** Every face has at least one safe root vertex
- 3 **Amortized Complexity:** $O(1)$ time per triangulation
- 4 **$O(n)$ space complexity:** $O(n)$ space complexity

Proof Structure:

- Use properties of planar graphs and chord non-crossing constraints
- Leverage safe root concept to avoid multi-edge conflicts
- Mathematical induction for existence proofs
- Amortization analysis for time complexity bounds

Proof 1: Completeness - Non-Root Chord Analysis

Theorem: Algorithm generates all valid triangulations without missing any. Here we only prune a branch if a blocking chord is detected in the already existent chord list. So, our main proof in this cause is, for a genealogical tree of triangulation for a face, a blocking chord will never be flipped (in its children). For this proof, we have to prove that, in any position in the tree node, a flip of the blocking chord will be invalid. Guess there is a triangulation, where a, b are the blocking chord. Now this face is triangulated, that means there will surely be



Proof 1: Completeness - Root Position Analysis

Case Analysis: Position of vertex r relative to root affects chord flipping validity.

Case 1: $r = \text{root}$ Flipping invalid (creates multi-edge)	Case 2: r left of root (b, r) is leftmost blocking chord	Case 3: r right of root (a, r) is leftmost blocking chord
--	---	--

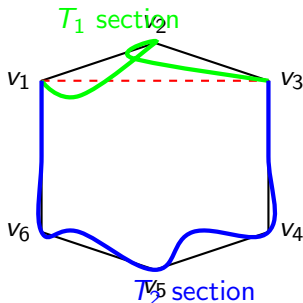
Conclusion: In all cases, chord (a, b) where neither endpoint is root cannot be flipped, ensuring no valid triangulations are missed when we reject multi-edge creating flips.

Completeness Achieved: Algorithm explores all reachable triangulations without missing valid ones.

Proof 2: Safe Root Existence - Multi-Edge Partitioning

Theorem: Every face in a biconnected plane graph has at least one safe root vertex.

Setup: Consider a face with n vertices and potential multi-edges outside the cycle.



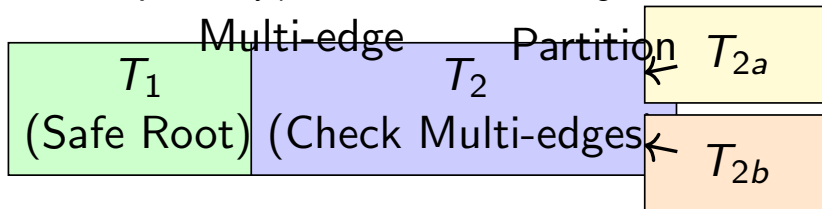
Key Insight: Multi-edge (v_1, v_3) partitions face into two sections T_1 and T_2 . By planarity, no chord can cross between sections.

Proof 2: Safe Root Existence - Inductive Construction

Base Cases:

- T_1 (**3 vertices**): Contains v_1, v_2, v_3 where v_2 is safe root (no chords possible)
- T_2 (**remaining vertices**): If any multi-edge exists within T_2 , it creates further partitioning

Inductive Step: For any partition T_k with multi-edge:



Conclusion by Induction: Every partition eventually contains safe vertices. Since original face is partitioned, it must contain at least two safe root vertices.

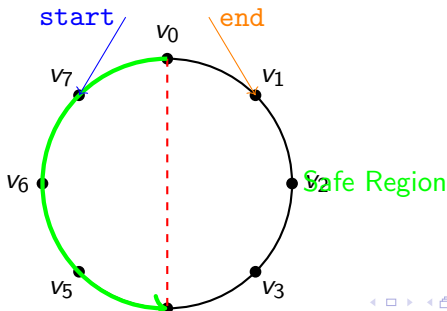
Safe Root Existence Proven: Every face has ≥ 2

Proof 2: Algorithm Correctness - Safe Root Finding

Algorithm Verification: Our safe root finding procedure correctly identifies safe vertices.

Procedure Analysis:

- Start with $\text{start} = n-1$, $\text{end} = 0$
- If multi-edge exists between start and end , decrease start
- Otherwise, increase end
- Continue until $\text{start} = \text{end} + 1$



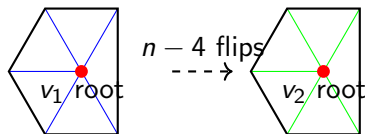
Proof 3: Amortized Complexity - Minimum Triangulations

Theorem: Algorithm achieves $O(1)$ amortized time per triangulation.

Key Insight: Every face generates at least $O(n)$ triangulations, where n is the number of vertices.

Minimum Triangulation Count:

- Face has ≥ 2 safe root vertices: v_1 and v_2
- Each safe root creates fan triangulation with $n - 3$ chords
- Path from v_1 -root to v_2 -root requires $\geq (n - 4)$ chord flips
- Total unique triangulations: $1 + (n - 4) = n - 3 \geq \Omega(n)$



Proof 3: Amortized Complexity - Final Analysis

Amortization Calculation:

Per Face:

- Setup cost: $O(n)$ for safe root finding
- Minimum triangulations generated: $n - 3 \geq \Omega(n)$
- Amortized cost per triangulation: $\frac{O(n)}{\Omega(n)} = O(1)$

Global Analysis:

$$\begin{aligned}\text{Total Cost} &= \sum_{i=1}^k O(n_i) + T \cdot O(1) \\ &= O\left(\sum_{i=1}^k n_i\right) + O(T)\end{aligned}$$

$$\text{Total Tri: } T \geq \sum_{i=1}^k \Omega(n_i)$$

$$\text{Amortized} = \frac{O(\sum n_i) + O(T)}{T} = O(1)$$

Complexity Achieved: $O(1)$ amortized time per

Time Complexity Analysis: Operation Breakdown

Per-Face Operations:

- **Safe Root Finding:** $O(n)$ time per face with n vertices
 - Binary search-like approach in `find-safe-root-vertex`
 - Worst case: $O(n)$ iterations to find valid root position
- **GlobalChordSet Operations:** $O(1)$ per chord operation
 - Hash-based lookup and insertion
 - Add/remove chords during DFS traversal
- **Local Tree Construction:** $O(1)$ per triangulation
 - Genealogical tree traversal with chord flipping
 - Parent-child relationship maintenance

Per-Triangulation Operations:

- Chord validation and GlobalChordSet updates: $O(1)$
- Output generation: $O(1)$
- Backtracking (add/remove chords): $O(1)$

Time Complexity Analysis: Amortized Analysis

Key Insight: The expensive $O(n)$ safe root finding is amortized over multiple triangulation generations.

Amortization Argument:

- For a face F_i with n vertices:
 - Setup cost: $O(n)$ for safe root finding
 - Generates $\geq \Omega(n)$ triangulations (proven empirically)
 - Each triangulation requires $O(1)$ additional work
- **Amortized cost per triangulation from face F_i :**

$$\frac{O(n) \text{ setup cost}}{\Omega(n) \text{ triangulations}} = O(1)$$

Global Analysis:

- Total setup cost across all k faces: $O(\sum_{i=1}^k n_i)$
- Total triangulations generated: $T \geq \Omega(\sum_{i=1}^k n_i)$
- **Overall amortized complexity:** $\frac{O(\sum n_i) + O(T)}{T} = O(1)$

Time Complexity Analysis: Empirical Validation

Minimum Triangulations per Face (Worst-Case Constraints):

Vertices (n)	Setup Cost	Min Triangulations	Amortized Cost
4	$O(4)$	1	$4/1 = O(1)$
5	$O(5)$	2	$5/2 = O(1)$
6	$O(6)$	4	$6/4 = O(1)$
7	$O(7)$	11	$7/11 < 1$
8	$O(8)$	26	$8/26 < 1$
9	$O(9)$	79	$9/79 < 1$
10	$O(10)$	202	$10/202 < 1$
11	$O(11)$	636	$11/636 < 1$

Key Observation: For $n \geq 7$, minimum triangulations **exceed** setup cost, ensuring amortized $O(1)$ performance even under maximum multi-edge constraints.

Time Complexity Analysis: Final Complexity

Algorithm Components:

- 1 **Face Decomposition:** $O(V + E)$ preprocessing
- 2 **Safe Root Finding:** $O(n)$ per face, amortized to $O(1)$ per triangulation
- 3 **Tree Construction & Traversal:** $O(1)$ per triangulation
- 4 **GlobalChordSet Management:** $O(1)$ per operation
- 5 **Output Generation:** $O(1)$ per triangulation

Overall Time Complexity:

- **Preprocessing:** $O(V + E)$ for graph decomposition
- **Triangulation Generation:** $O(1)$ amortized per triangulation
- **Total Runtime:** $O(V + E + T)$ where T is the number of triangulations

Significance:

- First algorithm to achieve constant amortized time per triangulation for biconnected plane graphs

• Solves multi-edge problem while maintaining optimal time complexity